

# MINDUEL LITE

An Interest-Based Two-Player Intellectual Competition Web Application

IU International University of Applied Sciences

Course: DLBSEPPSD01\_E — Software Development

Examination: Oral Project Report · Task 2 — Development of a Web Application

GitHub repository (title-slide link)

<https://github.com/MahmoudBKanaan/MiduelLite>

*Student name · Matriculation number · [replace before submission]*

# Outline — process over UI walkthrough

## Presentation focus (IU oral project report)

1. Task definition — problem, objective, users, requirements
2. Planning & process — Kanban, iterative-incremental model
3. Architecture & data — C4 containers, ER model, matchmaking
4. Implementation — state machine, core logic, security choices
5. Software result — operational demo evidence (not a screen tour)
6. Testing & Docker — automated tests, reproducible run
7. Reflection — goal achievement, limits, lessons, outlook

# 1. Task definition — problem & objective

## Problem

Short intellectual interaction is often blocked by registration, permanent profiles, and social features that are unnecessary for a brief two-player exchange.

## Objective (Task 2)

Deliver a demonstrable full-stack SPA: React frontend, Node/Express backend, PostgreSQL, two-user interaction, automated tests, Docker Compose, and documentation — optimised for submission reliability, not commercial scale.

# 1. Task definition — users & benefit

**Primary user:** person seeking a short, anonymous intellectual competition without permanent identity or PII.

## Derived requirements for the solution

- Zero signup friction → temporary session (playerId + sessionToken)
- Meaningful pairing → interest overlap matchmaking (no ML stack)
- Two-user interaction → turn-based peer-scored match state machine
- Demonstrable full stack → SPA + API + DB + Docker Compose

# 1. Task definition — MVP scope

*Scope reduction is an engineering decision aligned with the knowledge base and IU Task 2.*

| In scope (built)                   | Explicitly excluded          |
|------------------------------------|------------------------------|
| React SPA, Express API, PostgreSQL | Accounts / OAuth / JWT login |
| 4 routes; temporary sessions       | WebSockets, WebRTC audio     |
| Matchmaking 3→2→1 interests        | Chat, rankings, history UI   |
| Peer scoring, flags, results       | Cloud / K8s / microservices  |
| Tests, Docker Compose, docs        | AI scoring, admin CMS        |

## 2. Planning & process — Kanban

## Model: Iterative-Incremental Development managed with Kanban

- Why not pure Scrum? Solo student, fixed MVP, short timeline — ceremonies add overhead without team coordination benefit.
- Increments: concept → foundation → matchmaking → match engine → result → tests/docs → presentation.
- Board columns: BACKLOG → READY → IN PROGRESS → TEST → DONE (evidence below).

Figure 1. Kanban board snapshot (own project process evidence).

## 2. Planning — user journey (not a UI tour)

*Four screens map the planned flow; the oral report emphasises process and architecture around this journey.*

Welcome → Pool → Match → Result | Play again → Pool | Reset → Welcome

### Minduel Lite

Anonymous two-player intellectual competition



Display name  
Neo

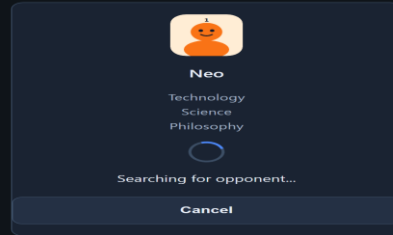
Avatar (choose one of 12)

Choose exactly 3 interests  
Selected 3 / 3

Technology Science Mathematics  
Programming Engineering Space Philosophy  
Psychology History Politics  
Economics Business Society Ethics  
Education Environment Health Law

Enter pool

Welcome

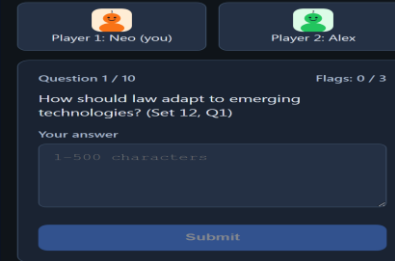


Neo  
Technology  
Science  
Philosophy

Searching for opponent...

Cancel

Pool



Player 1: Neo (you) Player 2: Alex

Question 1 / 10 Flags: 0 / 3

How should law adapt to emerging technologies? (Set 12, Q1)

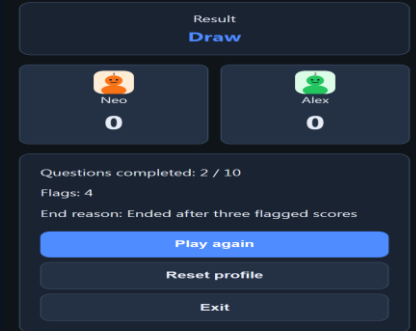
Your answer

1-500 characters

Submit

Match

### Match complete



Result  
Draw

Neo 0 Alex 0

Questions completed: 2 / 10  
Flags: 4  
End reason: Ended after three flagged scores

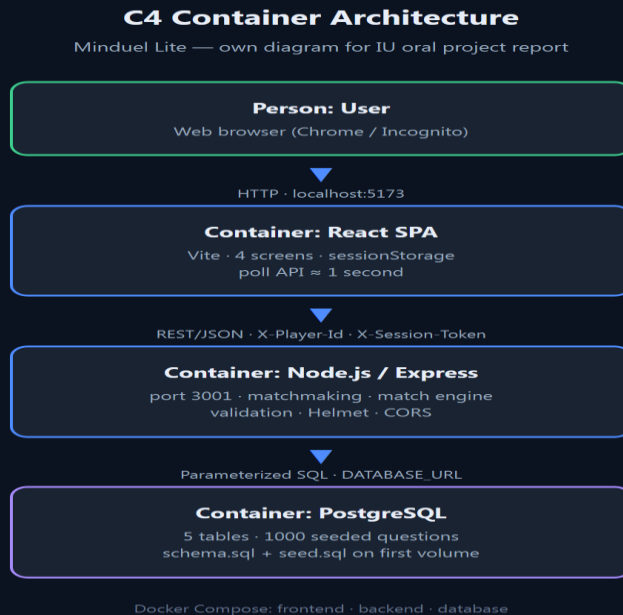
Play again  
Reset profile  
Exit

Result

*Figures 2–5. Application screenshots of the planned four-screen journey (own software).*

### 3. Architecture — C4 containers

*Design decision: one SPA, one API, one database — simplest structure that meets Task 2.*



*Figure 6. C4 container diagram (own diagram). Backend is authoritative for game state.*



# 3. Architecture — data model

## Entity-Relationship Model

PostgreSQL — five tables (own diagram)

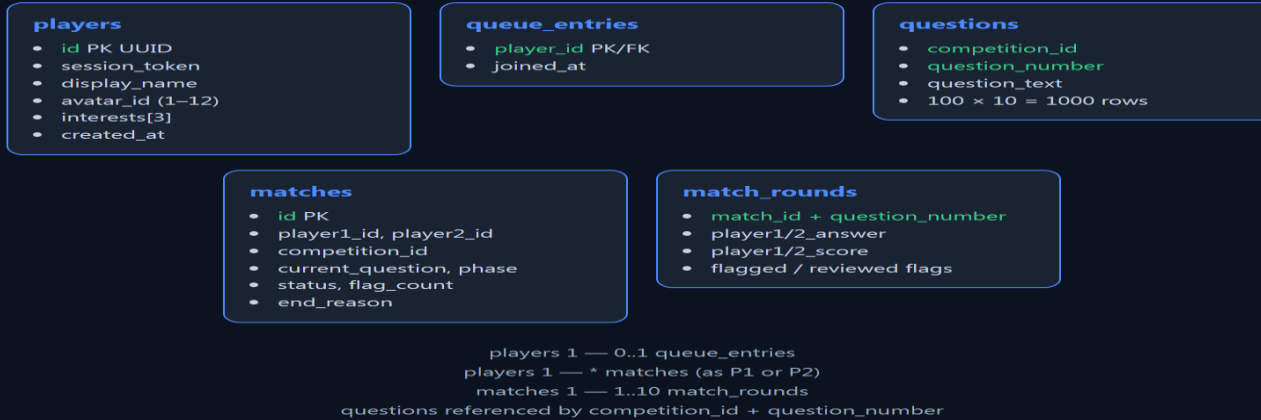


Figure 7. ER diagram (own diagram). Five tables; 100×10 = 1,000 seeded questions.

### 3. Architecture — matchmaking logic

$$\text{similarity} = \text{sharedInterestCount} / 3$$

| Shared | Similarity     | Matchmaking      |
|--------|----------------|------------------|
| 3      | 1.00           | Highest priority |
| 2      | $\approx 0.67$ | Next             |
| 1      | $\approx 0.33$ | Lowest eligible  |
| 0      | 0.00           | Never matched    |

#### Implementation choice

Set intersection only — no ML, no vector DB.

Waiting player = P1; joiner = P2.

Tie-break: earliest joined\_at.

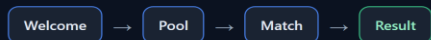
Excerpt: calculateInterestOverlap /  
selectBestCandidate

## 4. Implementation — state machine

### User flow & match state machine

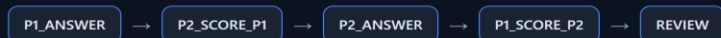
Minduel Lite — own diagram

#### Primary user flow

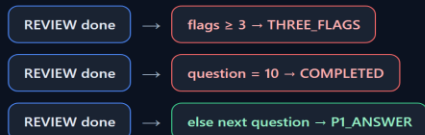


Play again → Pool (same session) · Reset / Exit → Welcome (clear sessionStorage)

#### Match phases (each question)



#### After both reviews



#### Matchmaking

`similarity = sharedInterestCount / 3`

Priority: 3 shared → 2 → 1 · zero shared = no match · earliest queue entry wins ties

### Backend authority

Frontend polls phase; does not invent rules.

resolveAfterBothReviews:

- $\text{flags} \geq 3 \rightarrow \text{THREE\_FLAGS}$
- $Q = 10 \rightarrow \text{COMPLETED}$
- else next question

advanceMatch() applies the decision in SQL.

Figure 8. User flow & match state (own diagram). Code: matchService.js.

# 5. Testing & reproducible run

Automated tests cover core business rules; Docker Compose is the only required runtime path.

## 197 — Successful automated tests

```
> minduel-lite-frontend@1.0.0 test
> vitest run

RUN v2.1.9 D:/Documents/IO/Minduellite/frontend
✓ tests/PoolPage.test.jsx (4 tests) 353ms
✓ tests/MatchPage.test.jsx (7 tests) 533ms
✓ tests/WelcomePage.test.jsx (4 tests) 1360ms
✓ WelcomePage (items 77-85) > PT-02 Enter Pool disabled until valid name, avatar, 3 interests 490ms
✓ WelcomePage (items 77-85) > Enter Pool POSTs player, saves session, navigates /pool 476ms
✓ tests/session.test.js (8 tests) 13ms
✓ tests/ResultPage.test.jsx (4 tests) 441ms
✓ tests/api.test.js (4 tests) 6ms

Test Files 6 passed (6)
Tests 31 passed (31)
Start at 18:21:49
Duration 7.11s (transform 296ms, setup 1.99s, collect 2.40s, tests 2.73s, environment 10.31s, prepare 1.30s)

--- BACKEND ---

> minduel-lite-backend@1.0.0 test
> vitest run

RUN v2.1.9 D:/Documents/IO/Minduellite/backend
✓ tests/core-mvp.test.js (17 tests) 930ms
✓ tests/api.test.js (10 tests) 776ms
✓ Player and matchmaking (DB) > DT-10/11 flagging and third flag ends match 316ms
✓ tests/review.test.js (10 tests) 937ms
✓ tests/result.test.js (9 tests) 418ms
✓ GET /api/matches/{matchId}/result (items 68-69) > returns result payload after THREE_FLAGS end 411ms
✓ tests/matchmaking.test.js (13 tests) 278ms
✓ tests/match-engine.test.js (9 tests) 426ms
✓ tests/security.test.js (11 tests) 91ms
✓ tests/players.test.js (12 tests) 125ms
✓ tests/foundation.test.js (3 tests) 45ms

Test Files 9 passed (9)
Tests 94 passed (94)
Start at 18:21:58
Duration 11.42s (transform 144ms, setup 0ms, collect 4.12s, tests 4.02s, environment 3ms, prepare 1.32s)
```

## 198 — Docker containers running

| NAME                   | IMAGE                | COMMAND                  | SERVICE  | CREATED       | STATUS                 | PORTS |
|------------------------|----------------------|--------------------------|----------|---------------|------------------------|-------|
| minduellite-backend-1  | minduellite-backend  | "docker-entrypoint.s..." | backend  | 3 minutes ago | Up 3 minutes           |       |
| minduellite-database-1 | postgres:16-alpine   | "docker-entrypoint.s..." | database | 3 minutes ago | Up 3 minutes (healthy) |       |
| minduellite-frontend-1 | minduellite-frontend | "docker-entrypoint.s..." | frontend | 3 minutes ago | Up 3 minutes           |       |

Command: docker compose up --build  
No manual DB setup (schema.sql + seed.sql).

Figures 9–10. Test run output; Docker Compose services (own environment).

## 5. Software result — what was demonstrated

*Evidence of an operational system (process outcome), not a feature-by-feature UI description:*

- Two independent browser sessions share one backend and database
- Interest matchmaking pairs compatible players into one matchId
- Authoritative phases: answer → score → review; flags and Q10 termination
- Results exclude flagged scores; play-again reuses temporary session
- Clean volume init: five tables + 1,000 questions without manual SQL steps
- Repository published with frontend, backend, DB, Docker, tests, docs

## 6. Evaluation — was the goal achieved?

Yes — for the defined academic MVP (not production readiness).

### Evidence

- Two anonymous browser sessions
- Interest matchmaking (3→2→1)
- 10-question completion + scoring
- Flags → THREE\_FLAGS termination
- Results exclude flagged scores
- SPA + API + Postgres integrate
- Docker Compose one-command run
- Automated tests pass (FE + BE)

### Limitations (intentional)

Excluded to protect scope and demo reliability: no accounts/production auth, polling not WebSockets, no disconnect recovery, subjective scoring, no moderation/history/audio, local Docker only, no production scale. Design choices — not unfinished MVP work.

## 6. Reflection & future improvements

### Lessons from this implementation

- Explicit exclusions reduced delivery risk
- Server-owned state simplified multiplayer consistency
- Polling was adequate for a two-player demo
- Focused tests beat high test volume

### Discussed only (not implemented)

- Authenticated accounts & history
- WebSockets / optional WebRTC audio
- Moderation & rankings
- Cloud deployment & stronger security ops

# Conclusion

- Task definition drove a deliberately small MVP
- Kanban + iterative increments structured solo development
- Architecture and state machine kept multiplayer consistent
- Tests, Docker, and GitHub complete the Task 2 evidence chain

<https://github.com/MahmoudBKanaan/MiduelLite>



# List of figures

---

*All figures below are the author's own work produced for this project (UI screenshots, process evidence, and original diagrams). External materials are listed only in the bibliography.*

Figure 1. Kanban board snapshot (project process evidence).

Figures 2–5. Application screenshots: Welcome, Pool, Match, Result (own software).

Figure 6. C4 container architecture diagram (own diagram).

Figure 7. Entity-relationship diagram (own diagram).

Figure 8. User flow and match state machine (own diagram).

Figure 9. Automated test run output (own development environment).

Figure 10. Docker Compose services status (own development environment).

# Bibliography

*APA 7th-style references for external sources used in the project and presentation. Original project diagrams and screenshots are not external sources.*

IU International University of Applied Sciences. (n.d.). Task for course DLBSEPPSD01\_E – Software Development [Examination task]. Examination Office.

IU International University of Applied Sciences. (n.d.). Guidelines for the creation of an oral project report. Examination Office.

Meta Platforms. (n.d.). React documentation. <https://react.dev/>

OpenJS Foundation. (n.d.). Express — Node.js web application framework. <https://expressjs.com/>

PostgreSQL Global Development Group. (n.d.). PostgreSQL documentation. <https://www.postgresql.org/docs/>

Docker Inc. (n.d.). Docker Compose documentation. <https://docs.docker.com/compose/>

Kanaan, M. B. (2026). Minduel Lite [Computer software]. GitHub. <https://github.com/MahmoudBKanaan/MiduelLite>